# Sudoku Solver using Backtracking

# Project Report

Name: Kunal

Registration number - 25BCE11146

Course: FUNDAMENTALS IN AI AND ML

Instructor: Dr.Vaishnavi S.

Submission Date: 26-03-2026

## 1 Introduction

Sudoku is a popular number puzzle that involves filling a grid while satisfying certain constraints. Solving it manually can take time and often involves trial and error. In this project, I implemented a Sudoku solver using a backtracking approach to automate the process.

## 2 Problem Statement

The goal of this project is to design a system that can solve a given Sudoku puzzle efficiently. The system should ensure that all Sudoku rules are followed while filling empty cells.

## 3 Approach

The problem is treated as a Constraint Satisfaction Problem (CSP). Each empty cell is considered a variable, and the possible values range from 1 to N depending on grid size.

The solution uses a backtracking (depth-first search) approach:

- Find an empty cell
- Try placing a valid number
- Continue recursively
- If a conflict occurs, backtrack and try another value

# 4 Implementation Details

The program is written in Python and includes:

- A function to find empty cells
- A function to check if a number can be placed
- A recursive solver using backtracking
- Input handling for user-defined puzzles

# 5 Features

- Solves 9x9 and 4x4 Sudoku puzzles
- Allows user input
- Displays board in readable format

# 6 Results

## 6.1 Input Puzzle

```
PS C:\Users\KUNAL\Documents\Python\AIML> python sudoku.py
Choose Sudoku size (perfect square only)
Enter size (4 or 9 recommended): 9

1. Use default puzzle
2. Enter your own
Choice: 2

Enter your 9x9 Sudoku (use 0 for empty)

Row 1: 5 3 0 0 7 0 0 0 0
Row 2: 6 0 0 1 9 5 0 0 0
Row 3: 0 9 8 0 0 0 0 6 0
Row 4: 8 0 0 0 6 0 0 0 3
Row 5: 4 0 0 8 0 3 0 0 1
Row 6: 7 0 0 0 2 0 0 0 6
Row 7: 0 6 0 0 0 0 2 8 0
Row 8: 0 0 0 4 1 9 0 0 5
Row 9: 0 0 0 0 8 0 0 7 9
```

### 6.2 Solved Output

```
You entered:

5 3 _ | _ 7 _ | _ _ _
6 _ _ | 1 9 5 | _ _ _
_ 9 8 | _ _ _ | _ 6 _
---------------------
8 _ _ | _ 6 _ | _ _ 3
4 _ _ | 8 _ 3 | _ _ 1
7 _ _ | _ 2 _ | _ _ 6
---------------------
_ 6 _ | _ _ _ | 2 8 _
_ _ _ | 4 1 9 | _ _ 5
_ _ _ | _ 8 _ | _ 7 9

Solved:

5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
---------------------
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
---------------------
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9
```

# 7 Challenges Faced

One of the main challenges was handling different grid sizes dynamically. Another issue was ensuring that the constraints were checked efficiently to avoid unnecessary recursion.

# 8 Conclusion

This project demonstrates how AI techniques like backtracking and constraint satisfaction can be applied to solve structured problems efficiently. It also helped in understanding how search algorithms work in practice.

# 9 What I Learned

- Practical use of backtracking algorithms

- How constraint satisfaction problems are solved
- Importance of breaking problems into smaller steps